Scientific Computing on Hardware

Subhodeep Chakraborty, Krishna Patil, Nara Prajwal and G.V.V. Sharma
Department of Electrical Engineering
IIT Hyderabad
Email: gadepall@ee.iith.ac.in

***Abstract*—In this paper, various algebraic and transcendental functions are implemented on microcontroller hardware. This is done by first modeling most functions using high school level differential equations, converting them to difference equations using elementary numerical techniques and then porting these algorithms to an ATMEGA328P microcontroller using AVR-GCC. In the process, we compare different numerical methods and finalize the Runge-Kutta (RK4) method for hardware implementation, based on numerical accuracy. We demonstrate the practical utility of this approach by building a scientific calculator based on the RK4 algorithm.**

***Index Terms*—Scientific Calculator, Microcontroller, Runge-Kutta Method, Quake III Algorithm, Shunting Yard Algorithm, Embedded C.**

I. Introduction

Physical systems are typically modeled as a system of ordinary differential equations (ODE) where the rate of change of a state variable y is a function of time t and the state itself: $\frac{dy}{dt} = f(t, y)$, given an initial condition $y(t_0) = y_0$. Most physical systems are now being implemented on digital platforms. This requires the design and implementation of algorithms capable of approximating complex functions under tight resource constraints. With respect to the ubiquity of differential equations as descriptors of real-world systems, the algorithms under the lens are those that perform or approximate numerical integration [1].

Numerical methods for function approximation on hardware often prioritize either simplicity or specialized efficiency. Euler's method [2] is the simplest to program, yet its first-order accuracy ($O(h)$) leads to significant numerical drift and instability unless excessively small step sizes are used. Second-order Runge-Kutta (RK2) methods improve precision but still feature much greater absolute error magnitudes than RK4 [3].

Alternative approximation strategies such as Taylor series or Maclaurin expansions are geometrically intuitive but often impractical for embedded systems [4]. This is because these methods require either the tedious recursive calculation of high-order derivatives, which are computationally expensive to evaluate, or for the coefficients for each function to be pre-stored, which results in memory usage piling up quickly for acceptable accuracy [5]. Also, the number of terms, and thus calculations needed for convergence, are much higher for Taylor series as compared to RK4 for the same accuracy [6]. In contrast, the RK4 framework achieves high-order accuracy without needing derivatives, instead evaluating the function at four strategically chosen intermediate points. This allows for a more robust and maintainable implementation when derivatives are complex or unavailable [7].

For specific transcendental functions, the CORDIC algorithm is highly efficient as it uses simple shift-and-add operations instead of multiplications [8]. However, CORDIC is limited by specific convergence ranges and requires different recursive sequences for different classes of functions [9]. Linear multistep methods like Adams-Bashforth can reduce function evaluations but are not self-starting and necessitate a queue of past history to be stored that increases memory usage [10].

More advanced structural equations and orthogonal collocation methods offer low levels of error but frequently involve complex and heavy computations or iterative non-linear solvers [11]. Similarly, adaptive methods like ode45 (used in MATLAB) provide excellent accuracy but introduce variable execution times that can cause unacceptable jitter in real-time environments, such as the those mentioned above [12].

A fixed-step RK4 approach is superior for predictable performance under the constraints discussed in this work, ensuring that calculations complete reliably within strict sampling windows, such as those in robotics or flight controllers [7]. Further, the single-step nature of RK4 is more suitable for limited microcontrollers as it avoids the RAM overhead of managing past solution data.

Despite extensive research into specialized hardware like FPGAs or complex applications such as GPS orbit computation and motor control, there is a lack of a mathematically unified framework for general scientific computing on 8-bit platforms [12]–[14]. Most related works treat different transcendental functions as separate implementation problems rather than a single unified mathematical structure [15].

This work bridges that gap by reformulating diverse scientific functions into high school-level ODEs, providing a robust, high-precision, and conceptually accessible framework for relatively limited microcontrollers. The proposed algorithms are then validated by implementing a scientific calculator using the ATMEGA328P.

II. Software Implementation

We first test the numerical algorithms in software before proceeding to hardware. Table I lists the functions implemented on the calculator while Table II lists the functions that can be formulated through ODEs [16].

TABLE I: Supported Functions and Operations

Category	Functions / Operations
Trigonometric	$\sin(x), \cos(x), \tan(x)$
Inverse Trigonometric	$\sin^{-1}(x), \cos^{-1}(x), \tan^{-1}(x)$
Exponential / Logarithmic	$e^x, \ln(x)$
Power / Root	x^y, $\sqrt{x}$
Factorial	$n!$
Basic Arithmetic	$+, -, \times, \div$
Constants	π, e
Input Symbols	Digits 0–9, decimal point, parentheses (,)

TABLE II: Implemented Functions and their ODEs

Function $y(x)$	Differential Equation	Initial Conditions
$\sin(x)$	$y'' + y = 0$	$y(0) = 0, \quad y'(0) = 1$
$\sin^{-1}(x)$	$\sqrt{(1-x^2)}y' - 1 = 0$	$y(0) = 0$
$\tan^{-1}(x)$	$(1+x^2)y' - 1 = 0$	$y(0) = 0$
$\ln(x)$	$xy' - 1 = 0$	$y(1) = 0$
e^x	$y' - y = 0$	$y(0) = 1$
x^w	$xy' - wy = 0$	$y(1) = 1$

A. RK4 based functions

The second-order differential equation for the sine function listed in Table II can be expressed as

$$\frac{dy}{dx} = z, \quad \frac{dz}{dx} = -y, \quad y(0) = 0, \; z(0) = 1 \tag{1}$$

The RK4 method then yields the update equation (see Appendix A).

$$\begin{aligned} k_1 &= hz_n, & l_1 &= -hy_n \\ k_2 &= h\left(z_n + \tfrac{l_1}{2}\right), & l_2 &= -h\left(y_n + \tfrac{k_1}{2}\right) \\ k_3 &= h\left(z_n + \tfrac{l_2}{2}\right), & l_3 &= -h\left(y_n + \tfrac{k_2}{2}\right) \\ k_4 &= h(z_n + l_3), & l_4 &= -h(y_n + k_3) \\ y_{n+1} &= y_n + \tfrac{1}{6}(k_1 + 2k_2 & z_{n+1} &= z_n + \tfrac{1}{6}(l_1 + 2l_2 \\ &\quad +2k_3 + k_4) & &\quad +2l_3 + l_4) \end{aligned} \tag{2}$$

where k_1, k_2, k_3, k_4 correspond to the RK-4 terms of the equation $\frac{dy}{dx} = z$ and l_1, l_2, l_3, l_4 corresponds to the RK-4 terms of the equation $\frac{dz}{dx} = -y$. The remaining functions in Table II are computed using a similar approach in Table III. The other functions/constants are computed using [16]

$$\cos x = \sin\left(\frac{\pi}{2} - x\right), \tan x = \frac{\sin x}{\cos x}, \cos^{-1}(x) = \frac{\pi}{2} - \sin^{-1}(x), \tag{3}$$

$$\pi = 4\tan^{-1}(1) \tag{4}$$

B. Non-RK4 Functions

1) The factorial is a simple recursive equation [17]

$$y_n = ny_{n-1} \tag{5}$$

2) The fast inverse square root algorithm (Quake III method) is used to efficiently compute $x^{-1/2}$ which is then used for computing $\sqrt{x}$. See Appendix B for details.

C. Expression Parsing

The Shunting Yard algorithm [18] is employed for converting infix expressions into postfix form (Reverse Polish Notation, RPN [19]) for evaluation. It relies on two structures

- A stack for operators/functions
- An output queue for the final postfix expression

The Shunting Yard Algorithm, together with RPN evaluation, respects operator precedence, associativity, and bracket handling, making it a cornerstone in expression processing across computing applications. Instructions for installation of a software based prototype for testing the algorithms are given in Appendix D.

III. Hardware Implementation

A. Hardware Required

TABLE IV: Materials required for the scientific calculator

Quantity	Component
25	Pushbuttons
1	LCD 16×2
1	Arduino Uno
–	Jumper wires
1	Potentiometer

- A button matrix for user input.
- An Arduino Uno microcontroller to process inputs and execute calculations.
- A 16×2 LCD connected to Arduino for showing results.
- Connections as shown in Fig. 2.

B. Circuit Connections

TABLE V: Arduino to LCD pin connections

Arduino Pin	LCD Pin	LCD Label	Description
GND	1	GND	Ground
5V	2	Vcc	Power Supply
GND	3	Vee	Contrast Control
D2	4	RS	Register Select
GND	5	R/W	Read/Write (fixed to Write)
D3	6	EN	Enable
D4	11	DB4	Data Bus (4-bit mode)
D5	12	DB5	Data Bus (4-bit mode)
D6	13	DB6	Data Bus (4-bit mode)
D7	14	DB7	Data Bus (4-bit mode)
5V	15	LED+	Backlight Anode
GND	16	LED-	Backlight Cathode

Function	k_1	k_2	k_3	k_4	y_{n+1}
Arcsine $\sin^{-1}(x)$	$\frac{h}{\sqrt{1-x_n^2}}$	$\frac{h}{\sqrt{1-(x_n+\frac{h}{2})^2}}$	$\frac{h}{\sqrt{1-(x_n+\frac{h}{2})^2}}$	$\frac{h}{\sqrt{1-(x_n+h)^2}}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Arctangent $\tan^{-1}(x)$	$\frac{h}{1+x_n^2}$	$\frac{h}{1+(x_n+\frac{h}{2})^2}$	$\frac{h}{1+(x_n+\frac{h}{2})^2}$	$\frac{h}{1+(x_n+h)^2}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Natural Log $\ln(x)$	$\frac{h}{x_n}$	$\frac{h}{x_n+\frac{h}{2}}$	$\frac{h}{x_n+\frac{h}{2}}$	$\frac{h}{x_n+h}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Power $y = x^w$	$h \cdot \frac{w}{x_n} y_n$	$h \cdot \frac{w}{x_n+\frac{h}{2}} \left(y_n + \frac{k_1}{2}\right)$	$h \cdot \frac{w}{x_n+\frac{h}{2}} \left(y_n + \frac{k_2}{2}\right)$	$h \cdot \frac{w}{x_n+h} (y_n + k_3)$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$

TABLE III: RK4 Update Equations for Single-Variable Functions

C. Button Matrix

The button matrix is a grid of push-buttons arranged in rows and columns. It allows multiple buttons to be connected to the microcontroller using fewer pins.

Working Principle:

- The Arduino scans the matrix by activating each row (setting it LOW) one at a time while reading the columns.
- When a button is pressed, it completes the circuit.
- By identifying the active row and column, the Arduino determines which button was pressed.
- Example: If second row is pressed when first column is active, the first column's second button is pressed.

This reduces the number of pins required to implement a calculator with 25 buttons.

TABLE VI: Key assignments in Mode 1 with Arduino pin mapping

	Col 1	Col 2	Col 3	Col 4	Col 5
Row 1	0	1	2	3	4
Row 2	5	6	7	8	9
Row 3	+	-	×	÷	$\sin(x)$
Row 4	$\cos(x)$	$\tan(x)$	$exp(x)$	$\ln(x)$	Clear
Row 5	Backspace	.	=	Mode shift	π

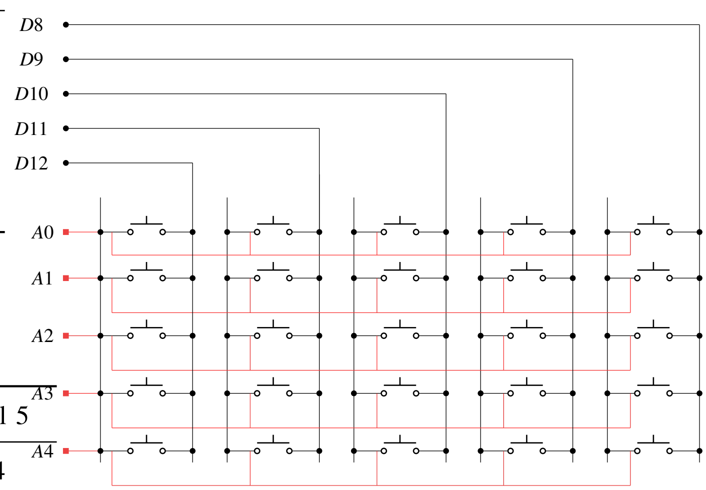
TABLE VII: Key assignments in Mode 2 with Arduino pin mapping

	Col 1	Col 2	Col 3	Col 4	Col 5
Row 1	0	1	2	3	4
Row 2	5	6	7	8	9
Row 3	(	)	x^y	fact(x)	$\sin^{-1}(x)$
Row 4	$\cos^{-1}(x)$	$\tan^{-1}(x)$	mod(x)	$\log_{10}(x)$	Clear
Row 5	Backspace	.	=	Mode shift	π

TABLE VIII: Keypad row/column connections to Arduino pins

Keypad Line	**Arduino Pin**
Row 1	D8
Row 2	D9
Row 3	D10
Row 4	D11
Row 5	D12
Col 1	A0
Col 2	A1
Col 3	A2
Col 4	A3
Col 5	A4

Fig. 1: Button Matrix



D. Circuit Schematic

The schematic for circuit connections is shown below,

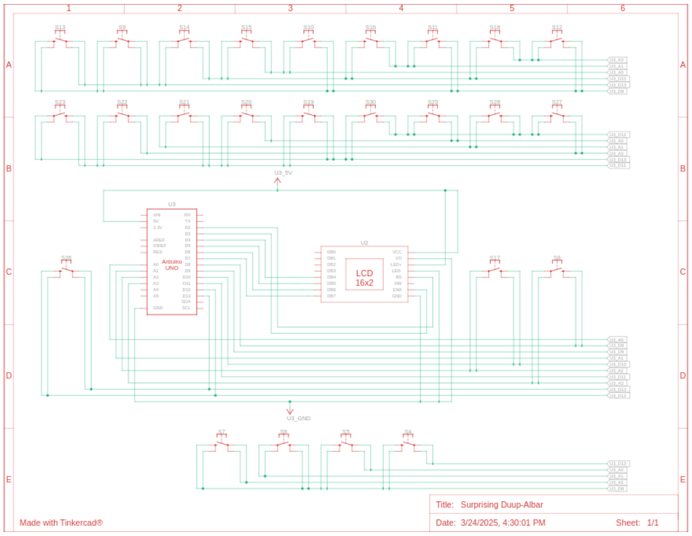


Fig. 2: Schematic of Circuit.

Appendix A
Runge-Kutta Method (RK4)

The RK4 method is a numerical technique to solve

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0 \tag{6}$$

with step size h. The next value is calculated as [2]

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{7}$$

where

$$k_1 = hf(x_n, y_n), \tag{8}$$
$$k_2 = hf(x_n + h/2, y_n + k_1/2), \tag{9}$$
$$k_3 = hf(x_n + h/2, y_n + k_2/2), \tag{10}$$
$$k_4 = hf(x_n + h, y_n + k_3). \tag{11}$$

Appendix B
Quake III Algorithm

The method obtains an initial approximation by manipulating the IEEE 754 floating-point representation of x, then refines it using Newton–Raphson iterations [20].

1) Bit manipulation with a "magic constant" ($0x5f3759df$) produces an initial guess y_0.
2) A single Newton–Raphson iteration dramatically improves accuracy.
3) An optional second iteration yields nearly full precision.

Mathematical Formulation: We want to approximate

$$y = x^{-\frac{1}{2}}. \tag{12}$$

Define

$$f(y) = \frac{1}{y^2} - x, \qquad f'(y) = -\frac{2}{y^3}. \tag{13}$$

Applying Newton–Raphson,

$$y_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)} \tag{14}$$
$$= y_k - \frac{\frac{1}{y_k^2} - x}{-\frac{2}{y_k^3}} \tag{15}$$
$$= y_k + \frac{y_k}{2}\left(1 - xy_k^2\right) \tag{16}$$
$$= y_k\left(\tfrac{3}{2} - \tfrac{1}{2}xy_k^2\right). \tag{17}$$

For the Quake III algorithm, $y_0 = A - (y_0 \gg 1)$, where the constant $A = 0x5f3759df$.

Thus one Newton–Raphson iteration is simply

$$y \leftarrow y_0\left(1.5 - 0.5\,x\,y_0^2\right). \tag{18}$$

Appendix C
Comparative Analysis of Numerical Methods

Three criteria were considered in the choosing of RK4 as the algorithm for a microcontroller base:

- Computing complexity
- Accuracy for the chosen precision level
- Memory usage

A. *Rejected Alternative Algorithms*

1) Euler's Method: The Euler method is the simplest numerical solver. The basic idea is to approximate the curve by its tangent line, meaning it is a first order solver.

- **Forward Euler (Explicit)** The slope at the current point is used as the approximation for the value at the end of the step.

$$y_{n+1} = y_n + h \cdot f(x_n, y_n) \tag{19}$$

- **Backward Euler (Implicit)** The slope at the end of the step is used as the approximation. y_{n+1} is present on both sides of the equation, so another algorithm is needed to separate it at each step.

$$y_{n+1} = y_n + h \cdot f(x_{n+1}, y_{n+1}) \tag{20}$$

- – **Drawback:** For a step size of h, the error is $O(h)$. Thus to reduce error, the step size would have to be increased drastically, which is computationally prohibitive.

2) Trapezoidal Rule: This is a second-order method approximates the integral by dividing the area under the curve into trapezoids instead of rectangles.

$$y_{n+1} = y_n + \frac{h}{2} \cdot (f(x_n, y_n) + f(x_{n+1}, y_{n+1})) \tag{21}$$

- – **Drawbacks:** The error is $O(h^2)$. It is also implicit, meaning each step would require another iterative solving mechanism such as Newton–Rhapson to separate the target variable.

Aside: Signal Processing Perspective: Another way to approach the previous formula is tranforming it through the s-domain and z-domain in succession, this reduces the work in solving of ODEs to simple algebraic manipulations. As an example, presented here is the solution to the *sin* function, whose ODE is:

$$y'' + y = 0 \tag{22}$$

1. **Laplace Transform:** Take the Laplace transform of the ODE (assuming zero initial conditions for the transfer function derivation):

$$s^2Y(s) + Y(s) = 1 \implies Y(s) = H(s) = \frac{1}{s^2 + 1}$$

2. **Bilinear Substitution:** Apply the Bilinear transform substitution $s \approx \frac{2}{h}\frac{1-z^{-1}}{1+z^{-1}}$, where h is the step:

$$H(z) = \frac{1}{\left[\left(\frac{2}{h}\frac{1-z^{-1}}{1+z^{-1}}\right)^2 + 1\right]}$$

3. **Algebraic Simplification:** Let $c = 2/h$, $A = c^2 + 1$. Simplify and group by powers of z

$$H(z) = \frac{(1 + z^{-1})^2}{c^2(1 - z^{-1})^2 + (1 + z^{-1})^2} \tag{23}$$

$$H(z) = \frac{1 + 2z^{-1} + z^{-2}}{c^2(1 - 2z^{-1} + z^{-2}) + (1 + 2z^{-1} + z^{-2})} \tag{24}$$

$$\frac{Y(z)}{X(z)} = \frac{1 + 2z^{-1} + z^{-2}}{(c^2 + 1) + (2 - 2c^2)z^{-1} + (c^2 + 1)z^{-2}} \tag{25}$$

$$Y(z) \cdot \left[1 + \frac{2 - 2c^2}{A}z^{-1} + z^{-2}\right] \tag{26}$$

$$= X(z) \cdot \frac{1}{A}\left[1 + 2z^{-1} + z^{-2}\right] \tag{27}$$

5. **Difference Equation:** Converting from z-domain to discrete time n (where $z^{-k}Y(z) \to y_{n-k}$):

$$y[n] = \underbrace{\frac{1}{A}(x[n] + 2x[n-1] + x[n-2])}_{\text{Input Terms (Source of Energy)}} \tag{28}$$

$$- \underbrace{\frac{2 - 2c^2}{A}y[n-1] - y[n-2]}_{\text{Feedback Terms (Oscillation)}} \tag{29}$$

Solving for the current term y_n after the impulse has passed:

$$y_n = \frac{8 - 2h^2}{4 + h^2}y_{n-1} - y_{n-2}$$

3) Series Expansion Methods: Taylor/Maclaurin series can also be used for this purpose.

- – **Drawbacks:** Precision cannot be acheived consistently as the number of terms needed vary with function and also with input value. As the input increases, convergence is slow, and number of terms needed to be stored in memory increases.

4) Second-Order Runge-Kutta Methods (RK2): A step up in accuracy from Euler's method are the second-order Runge-Kutta (RK2) methods. These methods use a preliminary slope calculation to make a more accurate step.

- **Heun's Method:** This method averages the slope at the beginning and end of the interval (using an Euler step as a predictor):

$$\tilde{y}_{n+1} = y_n + h \cdot f(x_n, y_n)$$
$$y_{n+1} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, \tilde{y}_{n+1}))$$

  – **Drawback:** The global error is $O(h^2)$. While this is a significant improvement over Euler's method, it is still much less accurate than the fourth-order RK4 method ($O(h^4)$). Given that RK4 only requires twice the number of function evaluations per step as these RK2 methods (four vs. two), the substantial gain in accuracy was judged as a highly favourable trade-off, as shown in Fig.3.

B. Justification for RK4 Selection

The fixed-step RK4 method was determined to be the optimal choice for this specific hardware implementation based on the following analysis (summarized in Table IX). The code for this analysis can be found at: https://github.com/gadepall/calculator/tree/main/analysis:

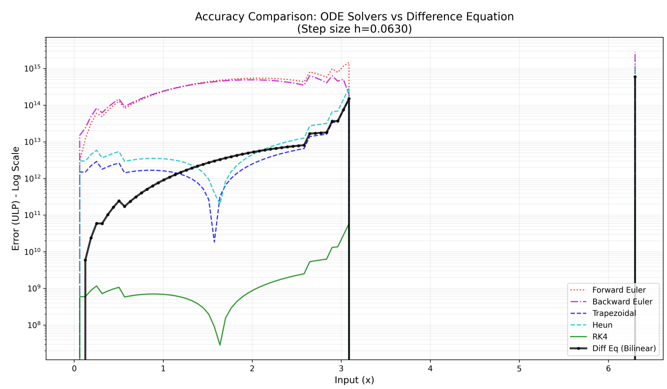


Fig. 3: Comparison of the accuracy of the algorithms

Here, each algorithm was compared against the gold-standard MPFR library up to 50 decimal places. Accuracy is represented using **Units in Last Place (ULP)** measure.

$$\text{Error}_{\text{ULP}} = \frac{|y_{\text{calc}} - y_{\text{true}}|}{\epsilon\,(y_{\text{true}})} \tag{30}$$

As per the results in Fig.3, RK4 demonstrates a clear dominance, with both maximum and average errors being many orders of magnitude less than the other algorithms. The Euler methods demonstrate an increasing drift from the true value with increase in input value, while the trapezoidal method, while stable in its error variation, featured much more absolute error than RK4. As expected, RK2 showed similar error variation as RK4 but much greater in magnitude.

TABLE IX: Accuracy comparison for sin(x) generation ($h \approx 0.06$)

Algorithm	Max Error (ULP)	RMS Error	Stability
RK4	2.38×10^{11}	3.44×10^{-7}	High
Trapezoidal	6.00×10^{14}	8.77×10^{-4}	High
Heun	1.20×10^{15}	1.74×10^{-3}	Moderate
Forward Euler	1.86×10^{15}	8.56×10^{-2}	Unstable
Backward Euler	2.84×10^{15}	7.27×10^{-2}	Unstable

Appendix D
Software Setup

A. Termux

1) On your android device, follow the instructions in https://github.com/gadepall/fwc-1 to setup and install Debian on Termux.

B. Platformio

1) Install Packages

```
apt install avra avrdude gcc-avr avr-libc
```

2) Follow the instructions in https://docs.platformio.org/en/stable/core/installation/methods/installer-script.html#super-quick-macos-linux to install platformio.
3) Execute the following on debian

```
cd ide/piosetup/codes
pio run
```

See Fig. 4 for the Arduino pin diagram.

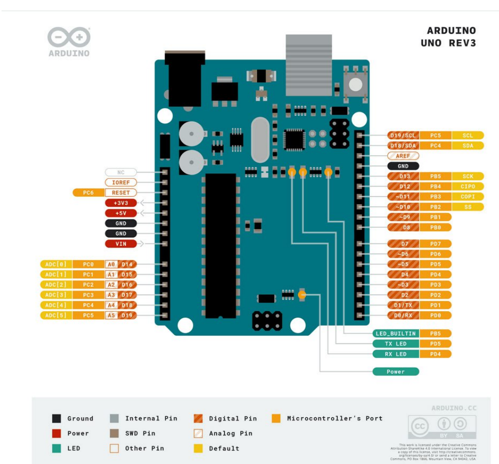


Fig. 4: Arduino Pinout

C. Arduino Droid

1) Install ArduinoDroid from apkpure
2) Open ArduinoDroid and grant all permissions
3) Connect the Arduino to your phone via USB-OTG
4) For flashing the bin files, in ArduinoDroid,

```
Actions->Upload->Upload Precompiled
```

then go to your working directory and select

```
.pio/build/uno/firmware.hex
```

for uploading hex file to the Arduino Uno
5) The LED beside pin 13 will start blinking

D. App Upload

1) Download the files present in https://github.com/gadepall/calculator/tree/main/software.
2) Compile the C files:

```
gcc -shared -o libcalc.so -fPIC func.c pars.c
```

3) Copy the app APK file `calci.apk` to the phone Internal Storage and install it from the File Manager.
4) Run the server in `proot-distro`:

```
python3 server.py
```

5) The app now functions as a scientific calculator.

E. Code Repository

The complete C implementations of the algorithms discussed in this paper are available at: https://github.com/gadepall/calculator/tree/main/codes.

```
codes
|-- defines.h
|-- func.c
|-- func.h
|-- gpio.c
|-- gpio.h
|-- keypad.c
|-- keypad.h
|-- lcd.c
|-- lcd.h
|-- main.c
|-- pars.c
|-- pars.h
|-- timer.c
`-- timer.h
```

Listing 1: Repository structure in codes/

References

[1] N. A. Udoh and U. P. Egbuhuzor, "Balancing precision and efficiency: Comparative analysis of numerical methods for initial value problems," *International Journal of Pure and Applied Mathematics Research*, vol. 5, no. 1, p. 61–69, Apr. 2025. [Online]. Available: http://dx.doi.org/10.51483/IJPAMR.5.1.2025.61-69

[2] B. S. Grewal, *Higher Engineering Mathematics*, 43rd ed. New Delhi, India: Khanna Publishers, 2014.

[3] N. Ahmad, S. Charan, and V. P. Singh, "Study of numerical accuracy of Runge-Kutta second, third and fourth order method," *International Journal of Computer and Mathematical Sciences (IJCMS)*, vol. 4, no. 6, pp. 111–117, June 2015. [Online]. Available: https://www.researchgate.net/publication/279849033_Study_of_Numerical_Accuracy_of_Runge-Kutta_Second_Third_and_Fourth_Order_Method

[4] G. Gadisa and M. A. Islam, "Comparison of higher order taylor's method and runge-kutta methods for solving first order ordinary differential equations," *Computing and Mathematical Sciences (Comp. & Math. Sci.)*, vol. 8, no. 1, pp. 12–13, January 2017. [Online]. Available: https://www.researchgate.net/publication/314100718_Comparison_of_Higher_Order_Taylor's_Method_and_Runge-_Kutta_Methods_for_Solving_First_Order_Ordinary_Differential_Equations

[5] A. E. Parker, "Runge-kutta 4 (and other numerical methods for ODEs)," Differential Equations, 7, 2021, accessed: 2025-02-09. [Online]. Available: https://digitalcommons.ursinus.edu/triumphs_differ/7
[6] N. A. Z. M. Noar, N. I. A. Apandi, and N. Rosli, "A comparative study of taylor method, fourth order runge-kutta method and runge-kutta fehlberg method to solve ordinary differential equations," in *THE 7TH BIOMEDICAL ENGINEERING'S RECENT PROGRESS IN BIOMATERIALS, DRUGS DEVELOPMENT, AND MEDICAL DEVICES: The 15th Asian Congress on Biotechnology in conjunction with the 7th International Symposium on Biomedical Engineering (ACB-ISBE 2022)*, vol. 3080. AIP Publishing, 2024, p. 020003. [Online]. Available: http://dx.doi.org/10.1063/5.0192085
[7] K. Murugesan, D. P. Dhayabaran, E. C. H. Amirtharaj, and D. J. Evans, "A fourth order embedded runge-kutta rkacem(4, 4) method based on arithmetic and centroidal means with error control," *International Journal of Computer Mathematics*, vol. 79, no. 2, p. 247–269, Jan. 2002. [Online]. Available: http://dx.doi.org/10.1080/00207160211925
[8] N. Eklund, "Cordic: Elementary function computation using recursive sequences," *The College Mathematics Journal*, vol. 32, no. 5, p. 330–333, Nov. 2001. [Online]. Available: http://dx.doi.org/10.1080/07468342.2001.11921899
[9] *AN5325: How to use the CORDIC to perform mathematical functions on STM32 MCUs*, STMicroelectronics, Mar. 2024, rev 6. [Online]. Available: https://www.st.com/resource/en/application_note/an5325-how-to-use-the-cordic-to-perform-mathematical-functions-on-stm32-mcus-stmicroelectronics.pdf
[10] G. K. Gupta, R. Sacks-Davis, and P. E. Tescher, "A review of recent developments in solving odes," *ACM Computing Surveys*, vol. 17, no. 1, p. 5–47, Mar. 1985. [Online]. Available: http://dx.doi.org/10.1145/4078.4079
[11] J. P. Allamaa, P. Patrinos, H. Van der Auweraer, and T. D. Son, "Safety envelope for orthogonal collocation methods in embedded optimal control," in *2023 European Control Conference (ECC)*, 2023, pp. 1–7.
[12] A. Akpunar Bozkurt, "Real-time sensorless speed control of pmsms using a runge–kutta extended kalman filter," *Mathematics*, vol. 14, no. 2, p. 274, Jan. 2026. [Online]. Available: http://dx.doi.org/10.3390/math14020274
[13] L. De Micco and H. A. Larrondo, "Fpga implementation of a chaotic oscillator using rk4 method," in *2011 VII Southern Conference on Programmable Logic (SPL)*. IEEE, Apr. 2011, p. 185–190. [Online]. Available: http://dx.doi.org/10.1109/SPL.2011.5782646
[14] S. A. Medjahed, "Introduction of the runge-kutta method in gps orbit computation," *International Journal of Engineering and Geosciences*, vol. 9, no. 2, p. 256–263, Jul. 2024. [Online]. Available: http://dx.doi.org/10.26833/ijeg.1403435
[15] L. Moroz, V. Samotyy, P. Gepner, M. Wegrzyn, and G. Nowakowski, "Power function algorithms implemented in microcontrollers and fpgas," *Electronics*, vol. 12, no. 16, p. 3399, Aug. 2023. [Online]. Available: http://dx.doi.org/10.3390/electronics12163399
[16] National Council of Educational Research and Training, *Mathematics: Textbook for Class XII, Part 1 and 2*. New Delhi, India: NCERT, 2006.
[17] G. V. V. Sharma, *C Programming in Middle School*.
[18] E. W. Dijkstra, "Algol 60 translation: An Algol 60 translator for the X1 and Making a translator for Algol 60," Mathematisch Centrum, Amsterdam, Tech. Rep. MR 35/61, November 1961. [Online]. Available: http://www.cs.utexas.edu/users/EWD/ewd00xx/EWD35.PDF
[19] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
[20] C. Lomont, "Fast inverse square root," Purdue University, Technical Report, February 2003. [Online]. Available: http://www.lomont.org/papers/2003/InvSqrt.pdf